LSDE Coursework

Ali Almalki

Part 2

Overview of The Application

WordFreq is a complete application designed using the Go programming language. Counting words in a text file is the basic feature of the programme. The top ten most common words in the document are returned. It has been migrated to AWS cloud using four main services connected with each other. The application runs on a virtual server (EC2 instance). Files are uploaded to AWS cloud via Amazon Simple Storage Service (Amazon S3). When the files are uploaded successfully to S3 bucket, an alert is sent to the job queue on Simple Queue Service (SQS) where the application processes the available messages. Another results queue holds the top ten results of the processed jobs. Eventually, all the output results are loaded into a NoSQL database table (DynamoDB).

Auto-Scaling Design Architecture

The purpose of Auto-Scaling architecture is to allow the application to scale up and down to meet demand. So, at this stage my goal was to support the application to handle greater loads. I did this by trying to vertically scale the application. For example, increasing the size of the instance from t2.micro to t2.large. Also, horizontally scaling by increasing the number of instances for the WordFreq application. When setting up the auto-scaling architecture for the WordFreq application, first I created Cloudwatch alarms attached to scaling policies in the auto-scaling group. The alarms' purpose is to be triggered once a threshold is breached to take the action of launching worker instances specified in the scaling policy. The scaling depends on the SQS jobs queue because that is where the messages are picked up by the consumer and get processed. The process here begins with uploading test files to S3 bucket, then, an event-notification sends an alert to the jobs queue indicating that there are available messages ready to be processed.

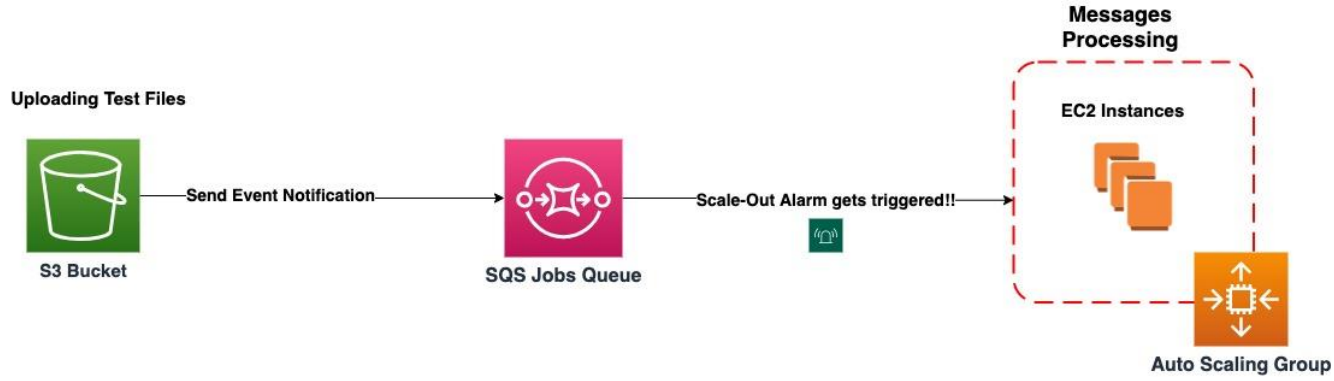


Figure 1: Jobs Queue Scaling Architecture.

I have specified the threshold that if the average number of visible messages on the jobs queue is equal to or more than three messages for one data point within a minute, the scale-out alarm will be triggered to launch three servers for processing, and if there are no more messages remaining on the jobs queue, the scale-in alarm will be activated to leave no instances running for cost saving.

In SQS service, we have a producer that sends messages to the SQS queue and a consumer that pull messages for processing. When defining the best metric for the auto-scaling infrastructure, I have chosen (ApproximateNumberOfMessagesVisible) metric because it represents the available messages for retrieval from the queue. An important point to notice is that the consumer has to delete the message after processing it. Amazon SQS sets a "*visibility timeout*" aiming to prevent other consumers from receiving and processing the message. The default visibility timeout for a message is thirty seconds to prevent other consumers from processing the same message again. During visibility timeout, the consumer has to delete the message after processing it or send a request to increase its timeout. If the consumer did not do one of these actions, the message would become visible again after the timeout expires. In order to set up Auto-Scaling Group (ASG), I had first to create a Launch Configuration that has the details of instances such as instance AMI, type, IAM role and Security Group attached to it.

To optimise the scaling operation, I have decided to launch type (t2.micro) instances because they are cheap and I also manually calculated its processing time: approximately 6 minutes to complete the whole processing task. In comparison, one instance of a powerful type such as (t2.2xlarge) is expensive and takes a long time for the processing: approximately 13 minutes. Using AWS Pricing Calculator, I have chosen three on-demand instance types (t2.micro, t2.medium and t2.2xlarge) to compare their monthly estimate costs. I have found that three (t2.micro) instances cost about $20.32, one (t2.medium) instance costs $25.17, and one (t2.2xlarge) instance costs $196.86 monthly. So I have decided to choose three on-demand (t2.micro) instances as they have the fastest processing time for the WordFreq application and are the most cost-effective of all instance types.

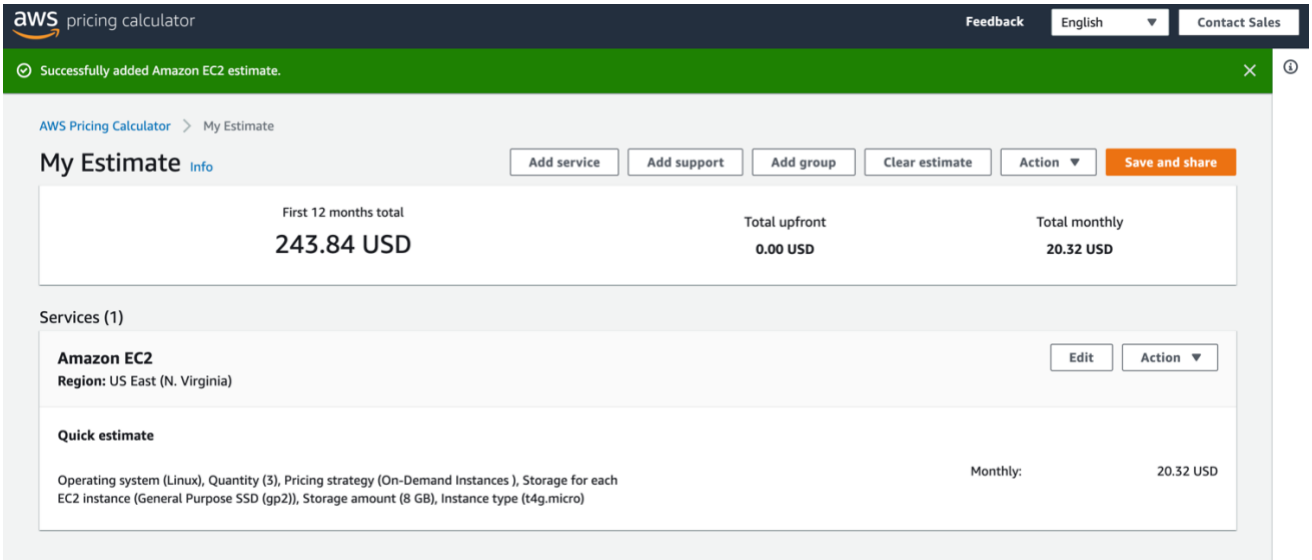


Figure 2: t2.micro Monthly Estimate Price

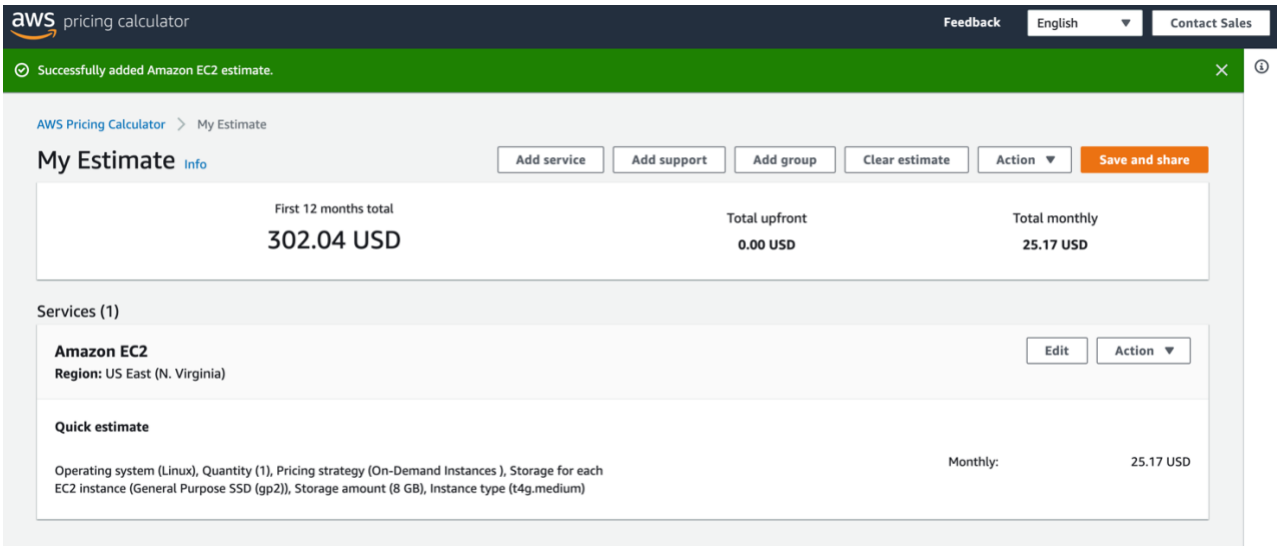


Figure3: t2.medium Monthly Estimate Price

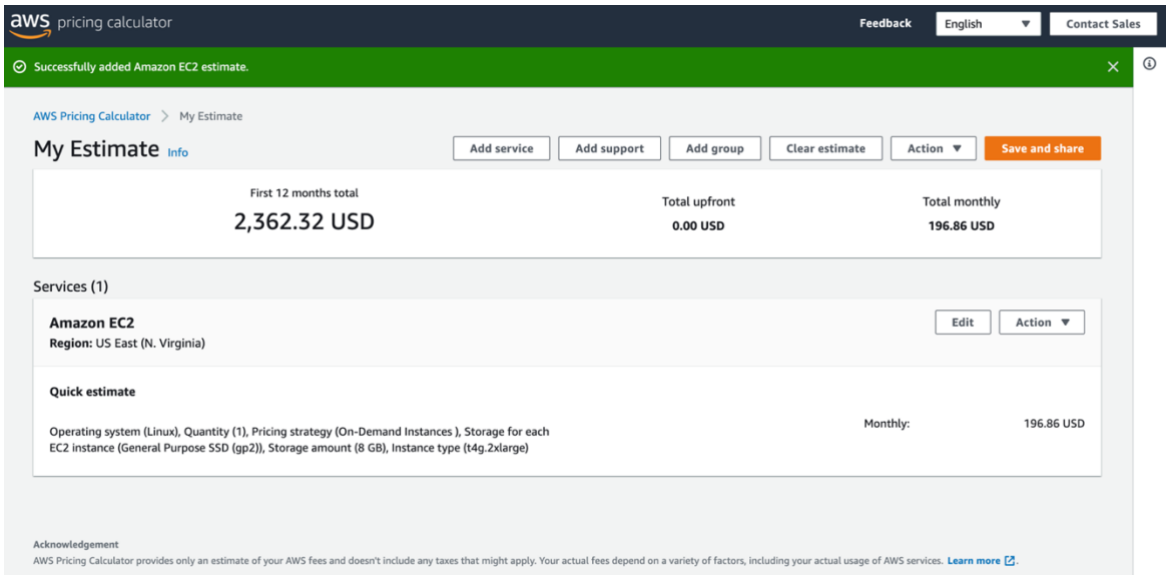


Figure 4: t2.2xlarge Monthly Estimate Price

Overview of The Testing Results

Once I set up my Auto-Scaling architecture, I monitored the scaling process, starting with the successful files upload to S3 bucket for testing. After that I noted, as shown in the screenshots images that the alarms are activated successfully, messages are processed on the SQS jobs queue and then sent to both results queue and DynamoDB. I also monitored what is happening in the Auto-Scaling group through the Activity History tab to know which alarms and actions have been activated.

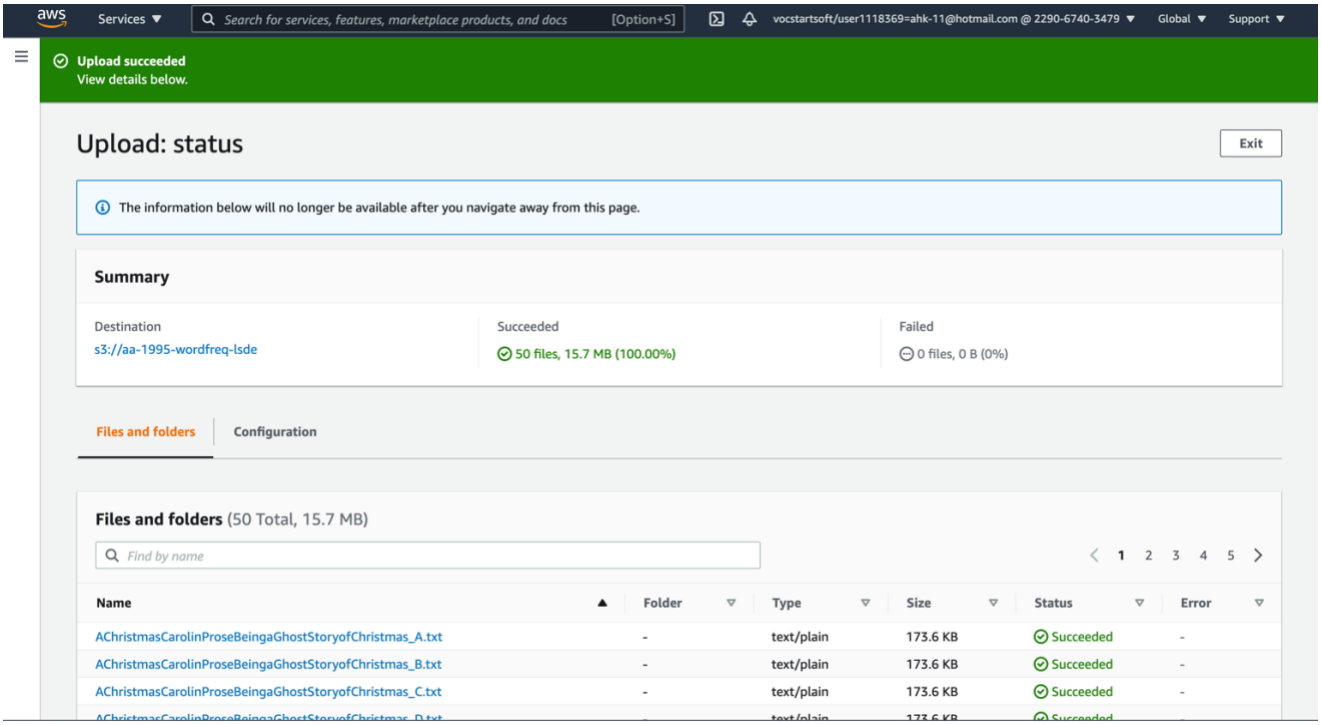


Figure 5: Uploading Test Files to S3 Bucket

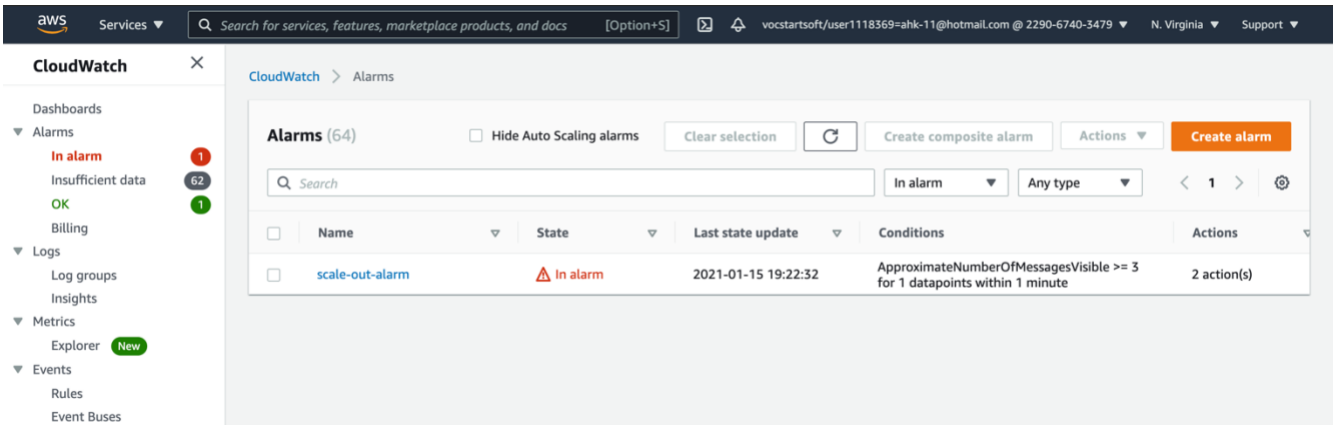


Figure 6: Scale-Out Alarm Response

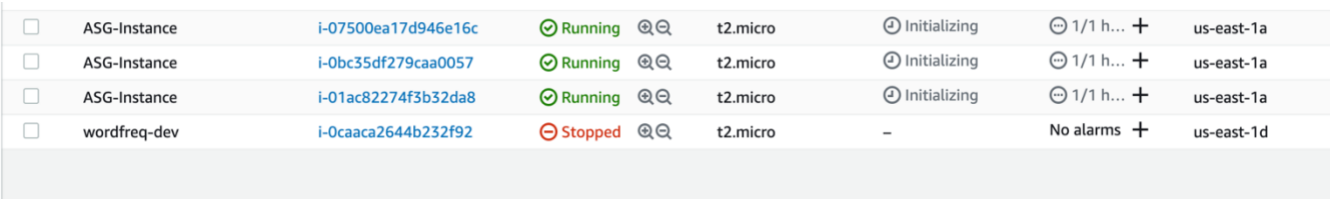


Figure 7: Auto-Scaling group worker instances status

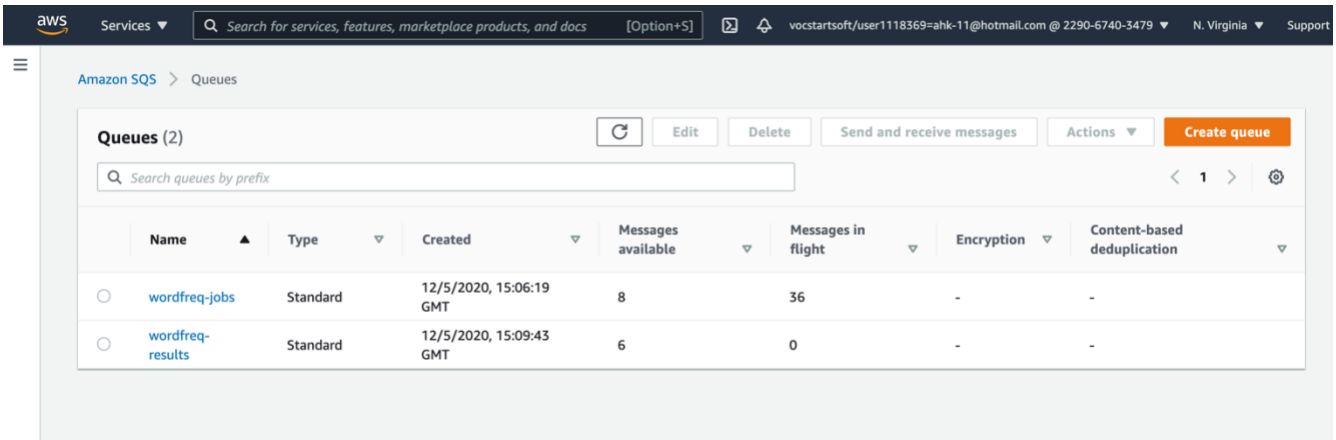


Figure 8: SQS Queues Status

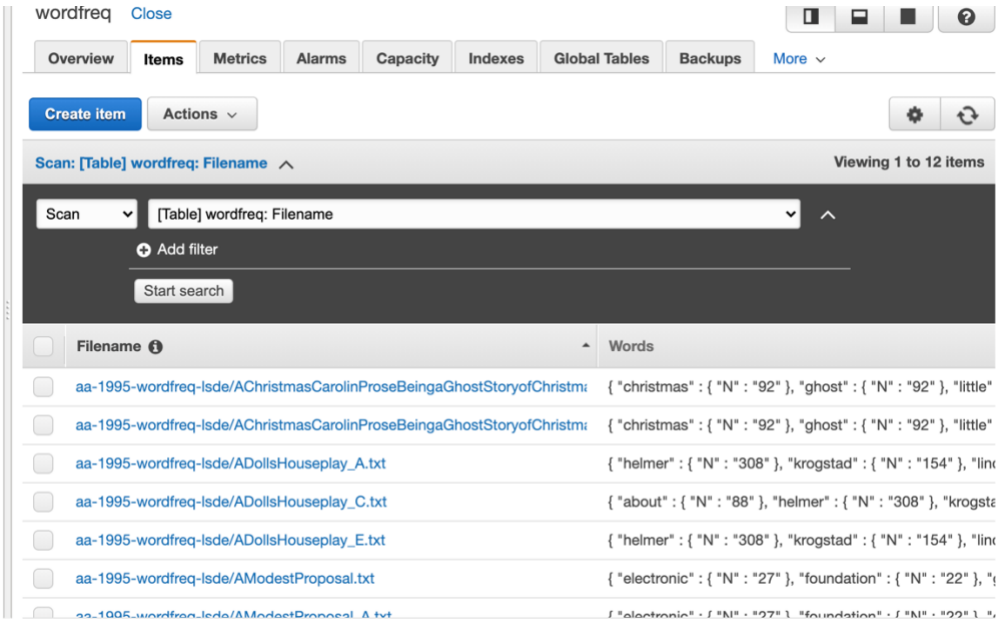


Figure 9: Saving the Results in DynamoDB

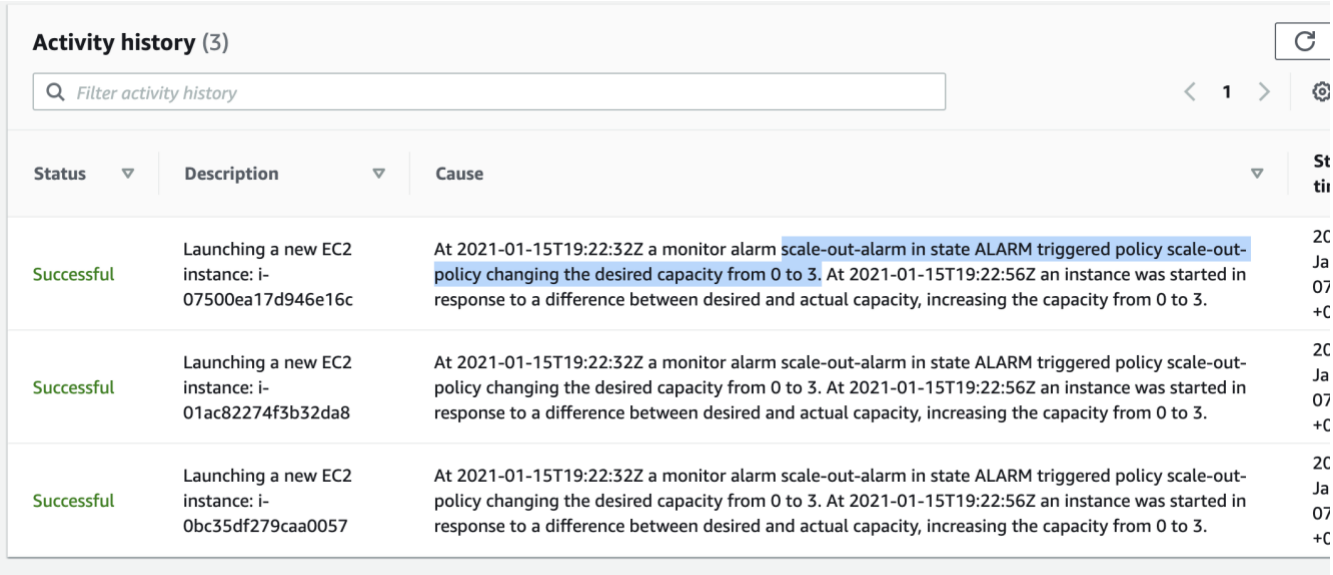


Figure 10: Auto-Scaling group Activity Monitoring

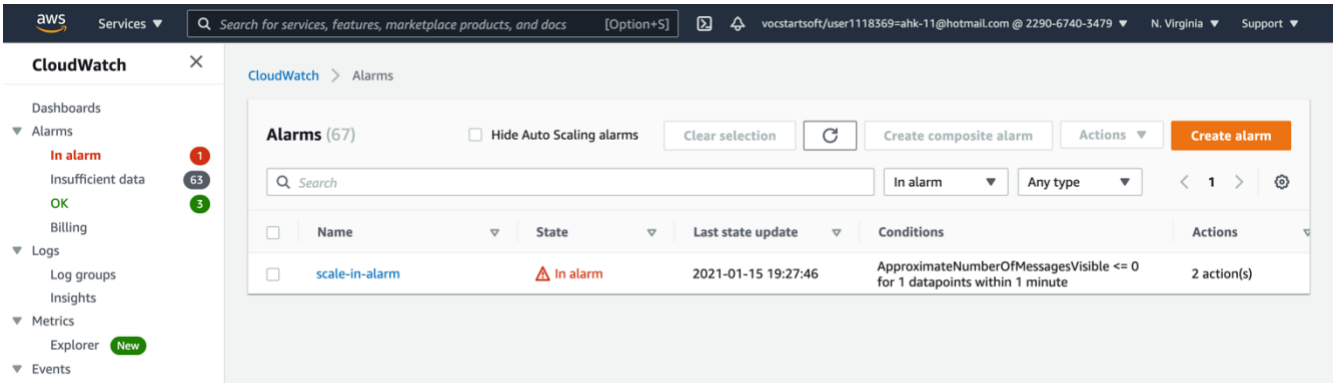


Figure 11: Scale-In Alarm Response

Securing and Optimising the WordFreak Architecture

To secure and optimise the WordFreq architecture, I have decided to create a new Virtual Private Network (VPC) called (wordfreq-vpc) to deploy my resources in it. This VPC has three Availability Zones with public and private subnets for each Availability Zone (AZ). I have chosen to do this because by launching instances in separate Availability Zones I can protect the WordFreq application from the failure of a single location. I have also set up public and private route tables and their purpose is to define access to the internet and between subnets.

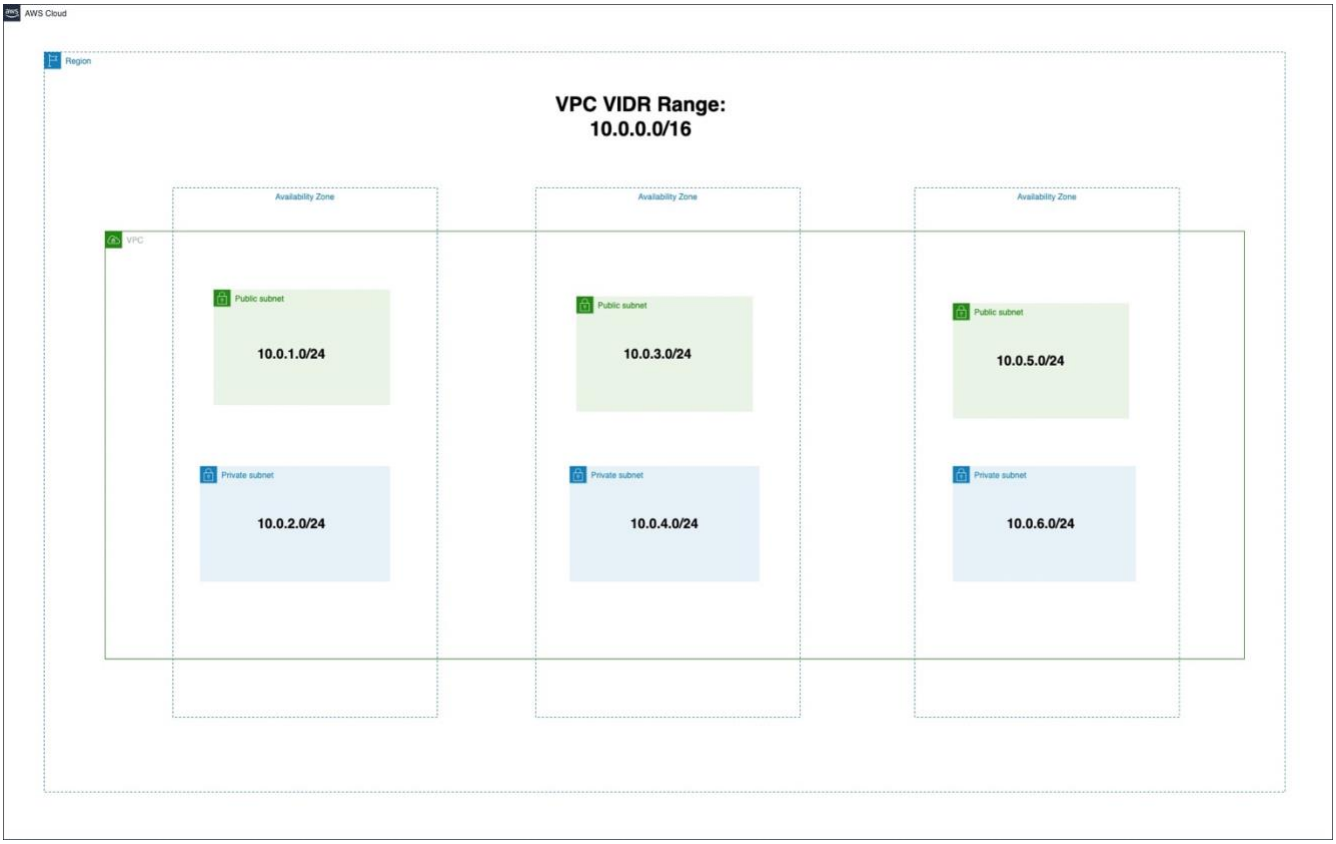


Figure 12: WordFreq Basic Network Architecture

I have also set up Network ACL which acts as a firewall that controls traffic from and to subnets. I have only allowed my IP address in the inbound rules to access the subnets so as to not make it open to the world. To protect the instances access, I have created a Security Group for it. Both Network ACL and Security Group control the traffic flow but the difference is that Network ACL is attached to the subnet level while security groups are attached to instances level.

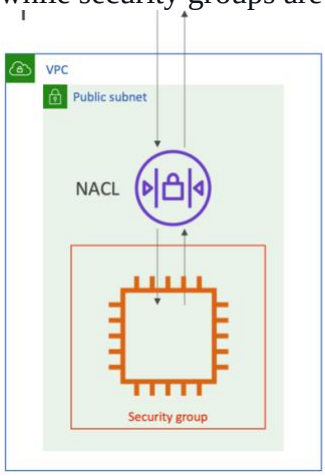


Figure 13: NACL Architecture

To protect the App data in the private subnet, I thought of setting up a NAT Gateway in the public subnet to allow the private subnet to access the internet while remaining private. I could not implement this feature due to limitations in the AWS Educate account, so instead I have set up a NAT instance in the public subnet to enable the worker instances in the private subnet to send traffic to the NAT instance in the public subnet. Internet traffic from the worker instances in the private subnet will be routed to the NAT instance, which then connects to the internet. The traffic is routed to the public IP address of the NAT instance.

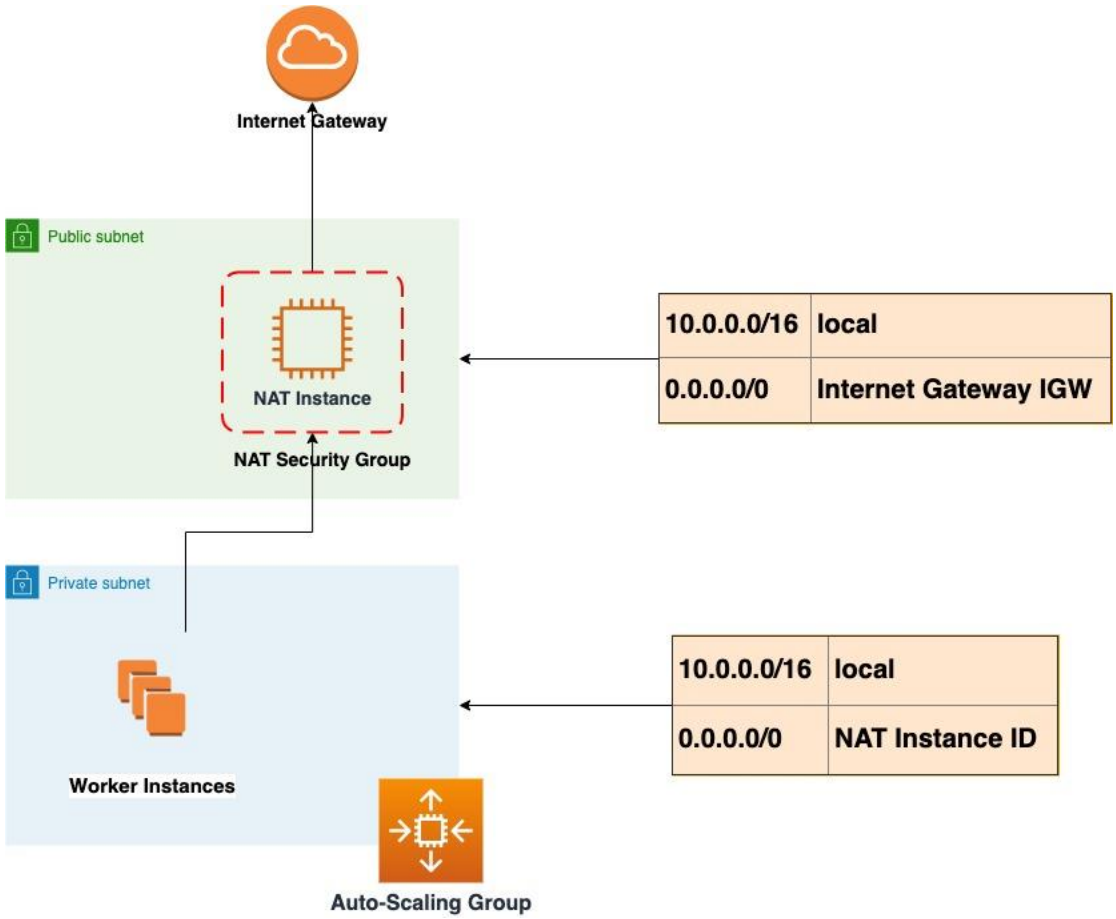
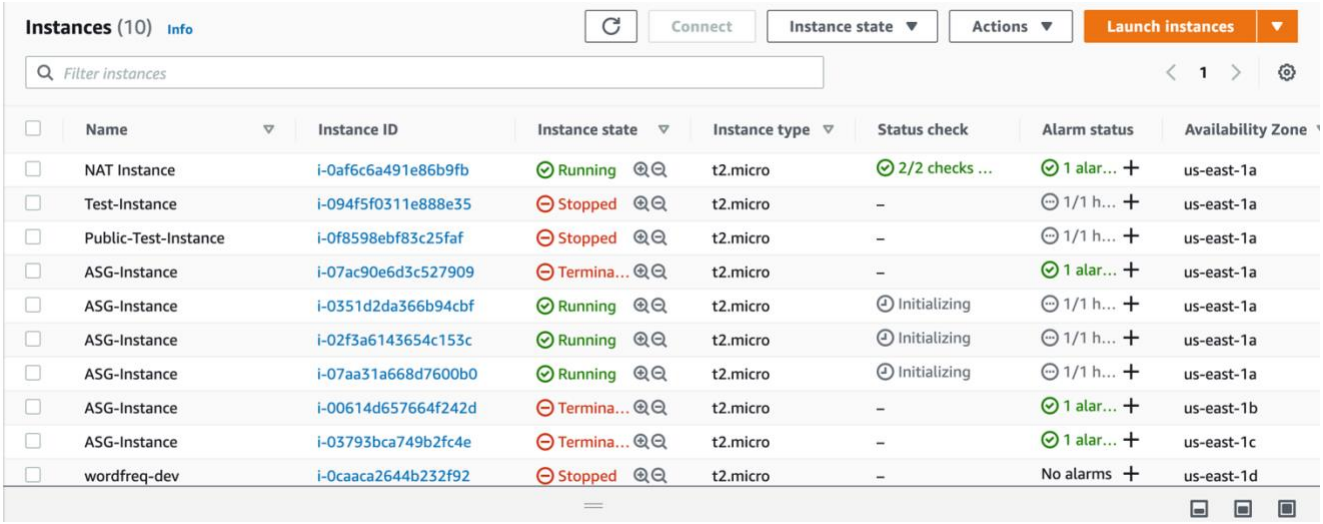


Figure 14: NAT Instance Architecture



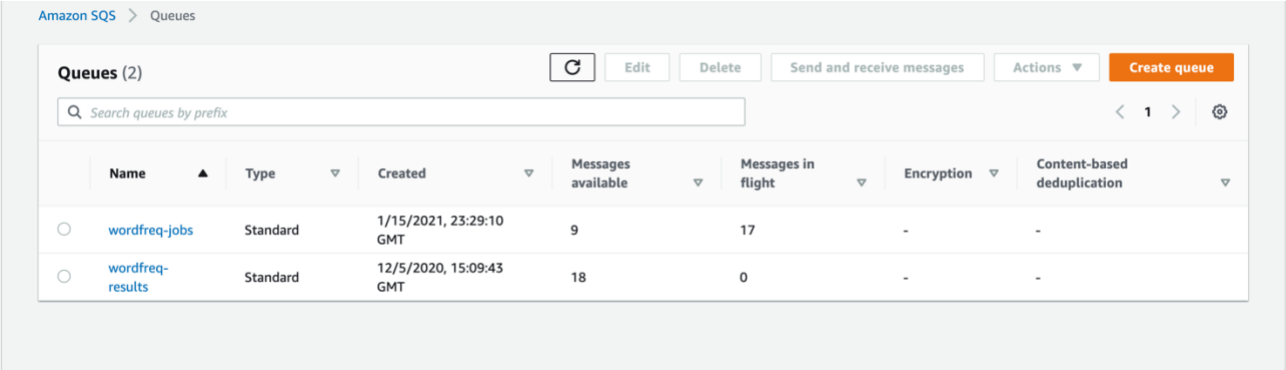
Instances (10) Info

Connect | Instance state | Actions | Launch instances

Filter instances

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
NAT Instance	i-0af6c6a491e86b9fb	Running	t2.micro	2/2 checks ...	1 alar...	us-east-1a
Test-Instance	i-094f5f0311e888e35	Stopped	t2.micro	–	1/1 h...	us-east-1a
Public-Test-Instance	i-0f8598ebf83c25faf	Stopped	t2.micro	–	1/1 h...	us-east-1a
ASG-Instance	i-07ac90e6d3c527909	Termina...	t2.micro	–	1 alar...	us-east-1a
ASG-Instance	i-0351d2da366b94cbf	Running	t2.micro	Initializing	1/1 h...	us-east-1a
ASG-Instance	i-02f3a6143654c153c	Running	t2.micro	Initializing	1/1 h...	us-east-1a
ASG-Instance	i-07aa31a668d7600b0	Running	t2.micro	Initializing	1/1 h...	us-east-1a
ASG-Instance	i-00614d657664f242d	Termina...	t2.micro	–	1 alar...	us-east-1b
ASG-Instance	i-03793bca749b2fc4e	Termina...	t2.micro	–	1 alar...	us-east-1c
wordfreq-dev	i-0caaca2644b232f92	Stopped	t2.micro	–	No alarms	us-east-1d

Figure 15: Auto-Scaling group worker instances status with NAT Instance



Amazon SQS > Queues

Queues (2)

Edit | Delete | Send and receive messages | Actions | Create queue

Search queues by prefix

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	1/15/2021, 23:29:10 GMT	9	17	-	-
wordfreq-results	Standard	12/5/2020, 15:09:43 GMT	18	0	-	-

Figure 16: SQS Queues Status

NAT instances are great for establishing a private connection to AWS services. However, deploying NAT instances in many AZs will not be cost-effective. So, I decided to do an alternative and more efficient solution which is creating VPC Endpoints to enable the instances in the private subnet to connect privately to AWS services and thus optimising the security of the App's data. VPC Endpoints are virtual devices that allow me to connect to AWS Services using a private network instead of a public www network. There will not be any need for Internet Gateway, NAT instance, virtual private gateway, VPN connection, or AWS Direct Connect for the connection which is a good advantage. Gateway VPC Endpoints for the services S3, DynamoDB are free while Interface VPC endpoint for SQS service costs $0.01 per hour, but still cheaper than a NAT gateway.

For the WordFreq architecture, I think VPC Endpoints are an optimal way because they enhance security and lower latency to access AWS services. I also needed to add them in the private route table in order to be able to route the traffic to the VPC Endpoints which then connects to the service.

When building a VPC Endpoint Interface for a service such as SQS, an elastic network interface (ENI) with a private IP address that serves as an entry point for service traffic is generated on the selected subnet.

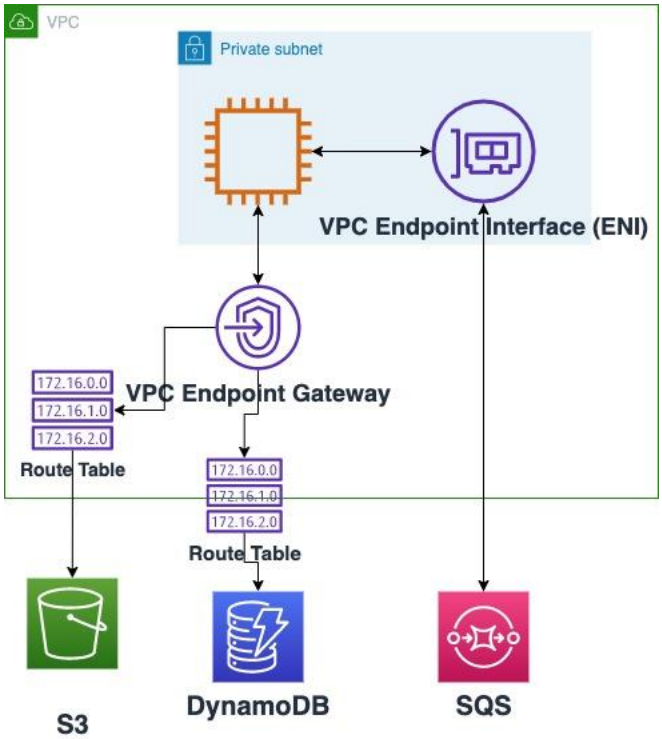
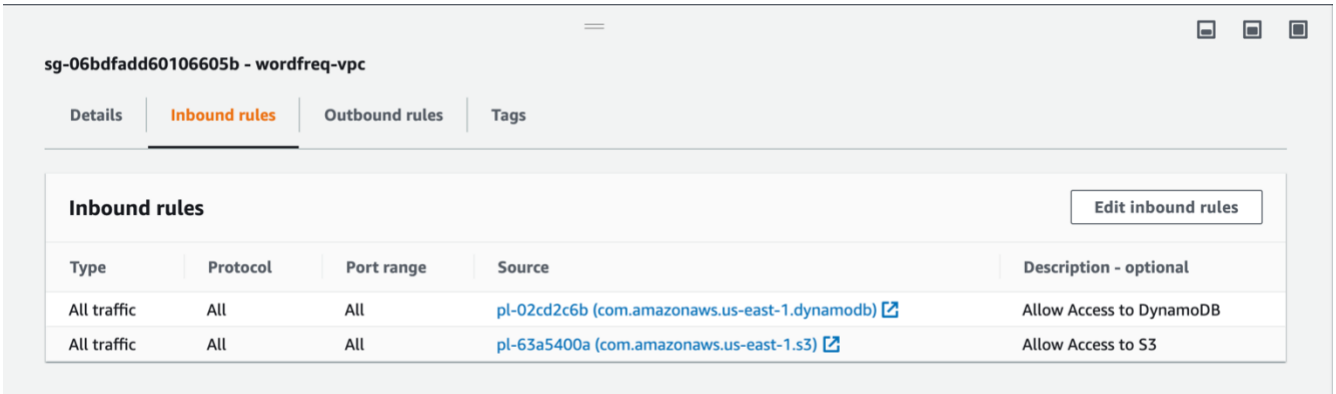


Figure 17: Changing the inbound rules for the wordfreq-vpc security group



sg-06bdfadd60106605b - wordfreq-vpc

Details | Inbound rules | Outbound rules | Tags

Inbound rules — Edit inbound rules

Type	Protocol	Port range	Source	Description - optional
All traffic	All	All	pl-02cd2c6b (com.amazonaws.us-east-1.dynamodb)	Allow Access to DynamoDB
All traffic	All	All	pl-63a5400a (com.amazonaws.us-east-1.s3)	Allow Access to S3

Figure 18: Changing the inbound rules for the wordfreq-vpc security group

Since the security group of the VPC is open to the world at port 22, it is not best practice to leave it like that. I had to restrict the access to make the application more secure. Therefore, I only enabled access for the VPC endpoints that I created to gain access to AWS services privately.

Issues

There are some issues I encountered during my implementation. When I created a launch configuration for the ASG to deploy the application in a new VPC environment, I had an error

saying that the security group is not linked to the VPC configured in the Auto Scaling group. I realised that it will not be possible to use a security group in one VPC in another VPC, so, I needed to solve this problem by creating a new Security Group dedicated to the new VPC that I created.

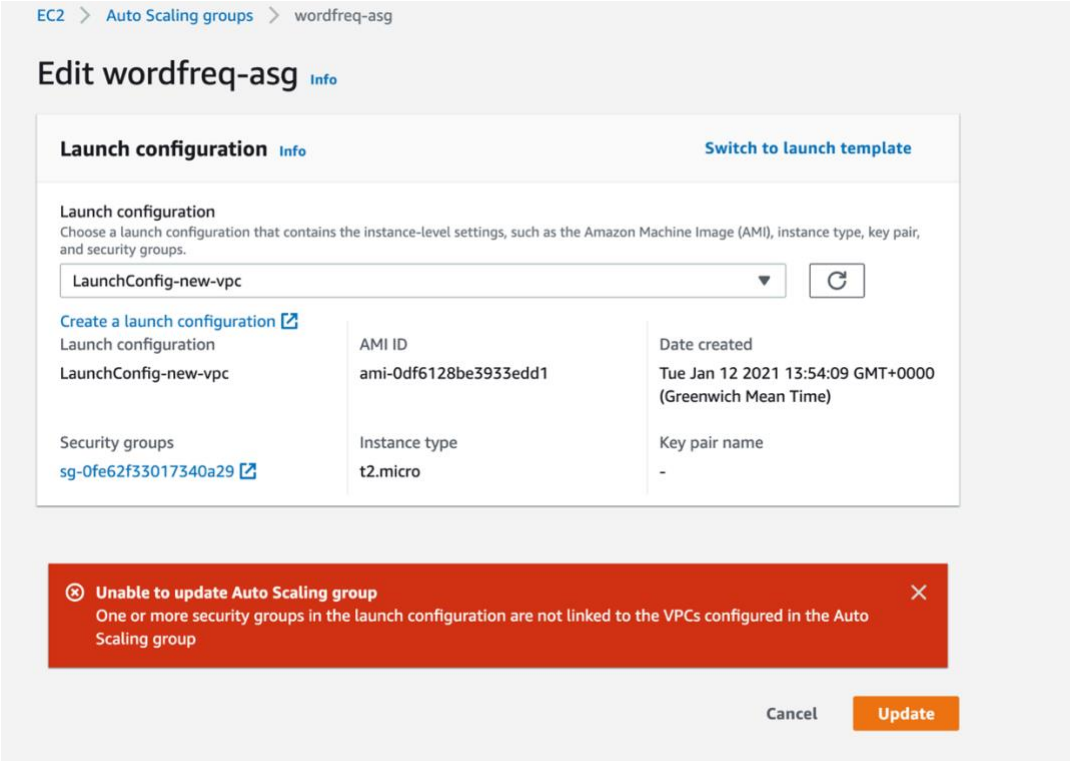


Figure 19: VPC Security Group Issue

Another issue arose when testing the new WordFreq architecture which, after I checked the instance logs, turns out to be I/O type issue with SQS operations. Basically, messages are added to the jobs queue successfully and Auto-Scaling group launches the worker instances in the private subnet but are not processing anything. I had to try to make changes with the Launch Configuration and Auto-Scaling group configurations and test again. Eventually, I solved this problem by editing the inbound and outbound rules of the security group attached to the VPC endpoints to allow all traffic to be able to connect to SQS service and start processing the messages. This attempt was successful and now the (wordfreq-asg-wordfreq-vpc) the Auto-Scaling group for the WordFreq VPC is functioning as expected in my Educate account. It launches three instances in the private subnets in all Availability Zones with a private connection to S3, DyanmoDB and SQS services through VPC endpoints, process the messages then terminate the worker instances.

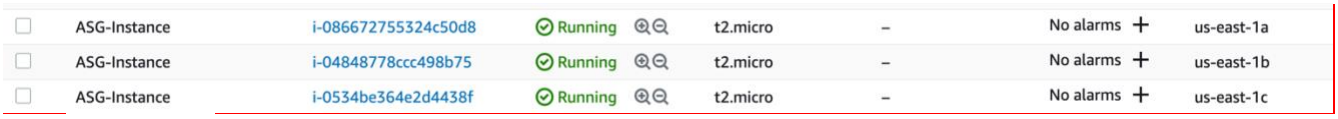


Figure 20: ASG Successfully Launches Worker Instances in Private Subnets in All Availability Zone

More Optimisations to Support the Current WordFreq Architecture

Regarding application data protection, using KMS (Key Management Service) to encrypt the data on all AWS services that I am using. KMS advantage evolves around managing cryptographic keys and controlling their uses across AWS services and applications. To protect the data at rest I can do this by using Client-Side encryption where the client, in our case is the WordFreq application running on EC2 instances, maintains the keys to encrypt the data and then sends the encrypted files into AWS storage S3, EBS and DynamoDB. The keys need to be rotated periodically to maintain the App's data security. Alternatively, I can use Amazon S3 Server-Side Encryption (SSE) where AWS manage encryption process for me. It is protected with the Advanced Encryption Standard (AES-256) which is a strong cipher that provides strong security. So, the objects are encrypted with a unique encryption key which then can be encrypted with a rotated master key. It stores the encryption keys and encrypted data in different hosts as an additional security feature. With Amazon S3 SSE I can create and apply bucket policies to only accept encrypted data to be uploaded to S3 bucket. Moreover, to manage the data on the S3 bucket effectively I can specify the storing duration of the objects in S3 bucket before deleting them.

Amazon Macie Service helps to manage data security and data privacy using machine learning and pattern matching to discover and protect sensitive data. It identifies sensitive data such as personally identifiable information (PII) and then sends alert about it.

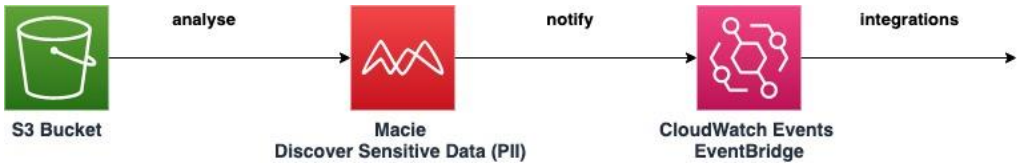


Figure 21: Amazon Macie Process

The only user for WordFreq application is myself, but if I wanted to create more users to access the App then Amazon Cognito service would come in handy because it provides authentication, authorisation and user management for the application. Basically, If I use this service, I can make the users sign in directly with a username and password or through a third-party application such as Facebook and Google. This service has a user pool which is users' directories that provide sign-up and sign-in options for the application users. It also has identity pool to enable me to grant users access to AWS services. Additionally, AWS Resource Access Manager, is a useful service to enable me to securely share AWS resources for WordFreq application with any AWS account or within any AWS organisation.

Regarding infrastructure protection, there are some good services. AWS Shield Service which provides protection from attacks such as SYN/UDP Floods, Reflection attacks and other layer 3/layer 4 attacks. Also, AWS Firewall Manager service that allow to centrally configure and manage firewall rules across AWS accounts and applications.

I can also enable Amazon Trusted Advisor for my account to check the AWS infrastructure and suggest best practices along with recommended actions. It is a way to give me advice related to

improving cost-optimisation, performance and security. If there is any security issue that should be taken care of, it will advise me. It will also show me if I am approaching the service limit. For example, the Educate account has a limit on how many EC2 instances may be launched. The recommended actions in Amazon Trusted Advisor are only suggestions and, while it is not necessary, it is a best practice to follow them. For instance, security groups in the architecture that have specific ports that allow unrestricted access to specific ports. As a result, I need to change it for better security.

Amazon Inspector is another service that generates automated security assessments for EC2 instances. It would be useful to install AWS Inspector Agent on the operating system of EC2 instances to analyse vulnerabilities and unintended network accessibility. Another issue is that anytime I have been doing some manual changes of the configurations of AWS resources, I did not have a list of all the things that happened, but I am able to obtain this using AWS Config service. AWS Config helps record configurations and changes over time and store this data into an S3 bucket to be recovered and analysed.

Further Improvements

While AWS services perform the work of processing the WordFreq application adequately, other data architectures for this processing task would be much more efficient and resilient. The testing files for the WordFreq application could be uploaded to a MapReduce framework, a programming model for processing and generating large data sets on a cluster. In MapReduce, the job has to be split down into a map and reduce phases. Then, the map phase will do the computation across all data items at the same time. This can be done by distributing all the data across multiple nodes stored in a cluster, and each node does the computation on its own data. The reduce stage would then take the map phase results, reduce it down into a single value and then send it back as the result of the computation. This kind of computation is highly scalable as it distributes data across many servers to be processed, and each node does part of the calculation independently.

Moreover, it is a very cost-effective solution as it operates on clusters of low-cost server hardware. Another benefit is that the method used for data processing in MapReduce is located on the same server, enabling faster data processing. Besides, It is highly parallel and fault-tolerant. If a single node fails, the duplicate data is always accessible on the other nodes, which can be used whenever required to ensure data availability.

Another alternative data processing architecture is Apache Kafka stream-processing software as it is also a more solid option for processing the data of the WordFreq application. Kafka is a distributed messaging system that allows decoupling data streams and systems and reduces the load on the WordFreq application server. The way Kafka works is by reading from an extensive range of data sources and grouping it into "topics". Each record sent to one group member (consumer groups). Messages are sent to Kafka broker, which is the server where the WordFreq app will be stored, to process and write the results to the database. Apache Kafka has many advantages in this case. It has an extremely high performance. The latency to exchange data from one system to another is usually less than ten milliseconds. This is called "real-time processing"

as the latency is very low. Additionally, Kafka software can scale to millions of messages per second. It is also has distributed, resilient architecture and is fault-tolerant.

References:

Amazon Inspector https://aws.amazon.com/inspector/

Amazon Macie https://aws.amazon.com/macie/

AWS Config https://aws.amazon.com/config/

AWS Trusted Advisor https://aws.amazon.com/premiumsupport/technology/trusted-advisor/

Cliff, D 2020, Week 8, Lecture 1 Google's Core Cloud Technologies, lecture slides, Large-Scale Data Engineering EMATM0051, University of Bristol, delivered December 2020.

Kafka https://www.bernardmarr.com/default.asp?contentID=1525

Identifying Opportunities to Right Size https://docs.aws.amazon.com/whitepapers/latest/cost-optimization-right-sizing/identifying-opportunities-to-right-size.html

NAT instances https://docs.aws.amazon.com/vpc/latest/userguide/VPC_NAT_Instance.html

Overview of AWS Security – Storage Services https://d1.awsstatic.com/whitepapers/Security/Security_Storage_Services_Whitepaper.pdf

Piper, B. and Clinton, D. (2019) AWS certified cloud practitioner study guide : clf-c01 exam. Indianapolis, Indiana: John Wiley & Sons. p.304.

Route Tables https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

SQS visibility timeout https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html

VPC Endpoints https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

What is MapReduce? https://www.educba.com/what-is-mapreduce/

When to Avoid Cloud Computing, 28 February 2017, Techgenix Website http://techgenix.com/when-to-avoid-cloud-computing/